Name: Opeyemi Caleb Akoki

Student ID: 16980763

Module: Software Security 7021SCN

CRITICAL REFLECTION ON 7021SCN SOFTWARE SECURITY PORTFOLIO

This portfolio applies practical security techniques across five tasks covering vulnerability discovery, threat modelling, static analysis, dynamic testing, and supply chain compliance. Each maps to the NIST SSDF (SP 800-218), which organises secure development into four pillars: Prepare the Organisation (PO), Protect the Software (PS), Produce Well-Secured Software (PW), and Respond to Vulnerabilities (RV).

SSDF PS.1 - Protect Software from Tampering

This repository has two automated pipelines. `.github/workflows/sast.yml` runs Semgrep and Bandit on every push, and `.github/workflows/generate-sbom.yml` generates the SBOM using Anchore Syft on a fresh GitHub-hosted Ubuntu runner. See Figure 1.

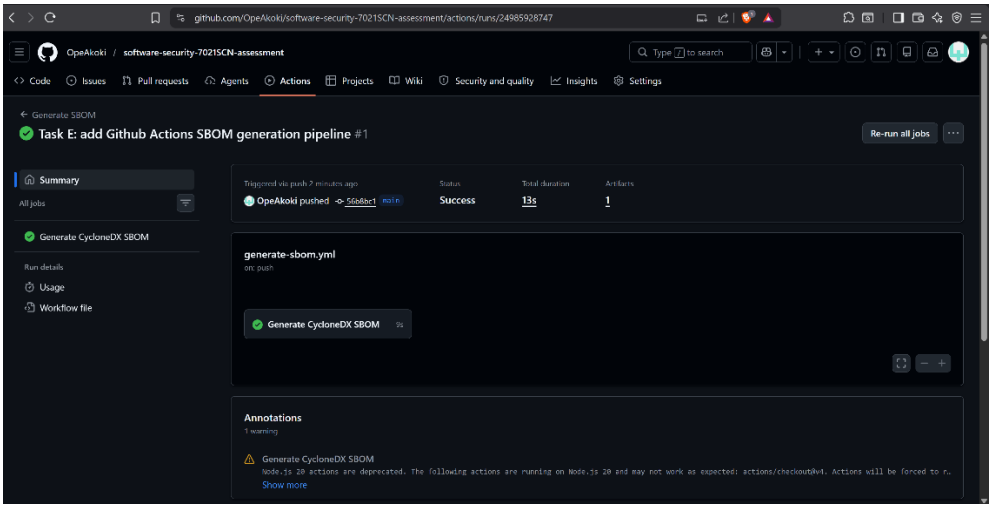


Figure 1: GitHub Actions - Generate SBOM pipeline run (PS.1 evidence)

```
on:
  push:
    branches: ["main"]
```

```
jobs:
  sbom:
    runs-on: ubuntu-latest
```

The SBOM is never generated on a developer's machine. Every push spins up a clean Ubuntu environment, runs Syft, uploads the artefact, and destroys the machine. No human is involved between the commit and the output, removing any possibility of tampering and directly satisfying PS.1.

SSDF PW.4 - Maintain Provenance Data

```
570 {
571   "bom-ref": "pkg:pypi/pip@24.0?package-id=5fa4075dce96
572   "type": "library",
573   "author": "The pip developers <distutils-sig@python.or
574   "name": "pip",
575   "version": "24.0",
576   "licenses": [
577     {
```

Figure 2: sbom.cdx.json - pip dependency block showing exact version (PW.4 evidence)

The sbom.cdx.json file records every Python package and its exact version. When a zero-day is disclosed, a team without this file spends days checking every server manually. A team with it runs one query and has an answer in seconds, including nested transitive dependencies. This captures strict provenance records for every component as required by PW.4.

SSDF RV.1 - Identify and Confirm Vulnerabilities on an Ongoing Basis

The pipeline generates the SBOM in CycloneDX JSON format. Grype ingested this file and returned two findings with GHSA identifiers and severity ratings. See Figure 3.

```
(venv) userskokey@Opeyemi:/mnt/c/Users/USER/software-security-7021SCN-assessment$ dock
er run --rm -v $(pwd)/task-e-SBOM/sbom:/sbom anchore/grype sbom:/sbom/sbom.cdx.json
NAME   INSTALLED   FIXED IN   TYPE    VULNERABILITY        SEVERITY   EPSS          RISK

pip    24.0        25.3       python  GHSA-4xh5-x5gv-qwph  Medium     < 0.1% (5th)  < 0.1

pip    24.0        26.0       python  GHSA-6vgw-5pg2-w6jp  Low        < 0.1% (7th)  < 0.1
(venv) userskokey@Opeyemi:/mnt/c/Users/USER/software-security-7021SCN-assessment$
```

Figure 3: Grype scan output against *sbom.cdx.json* - two pip vulnerabilities identified (RV.1 evidence)

A standardised CycloneDX JSON file can be ingested by tools like Grype, Aqua Trivy, and enterprise SIEMs, which cross-reference every component against live CVE databases without human involvement. Formatting the SBOM as machine-readable data allows vulnerability scans to run continuously, satisfying RV.1.

Threat Modelling and SDLC – Task B

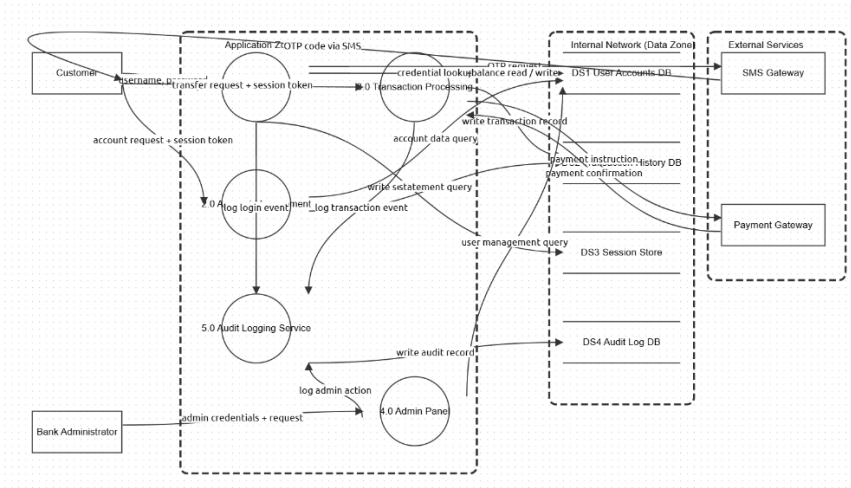


Figure 4. NovaTrust Bank Mobile Application - Threat Dragon DFD with three trust boundary zones (Task B evidence)

STRIDE was applied to the NovaTrust Bank Mobile Application across eleven threats mapped to a DFD with three trust boundary zones. STRIDE surfaces design-level flaws that no scanner can detect. The cookie forgery vulnerability has no code pattern to find. It only appears when asking the Spoofing question: who can pretend to be someone else? Its limitation is that STRIDE identifies threat categories but does not rank by likelihood or business impact, meaning a theoretical low-probability threat can receive the same attention as a near-certain critical one. Applying it at the design stage ensures security requirements are defined before any code is written.

Dynamic Analysis & Exploit Development – Task D

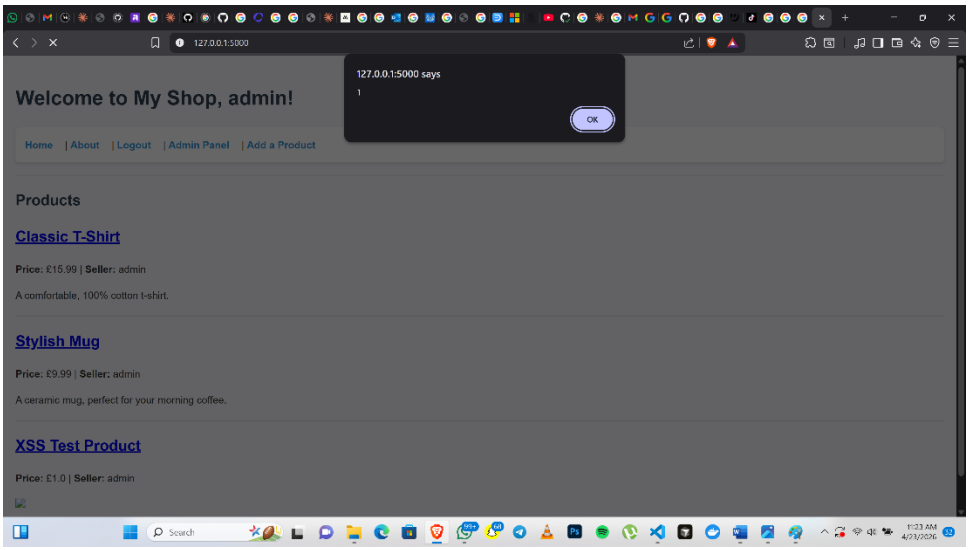
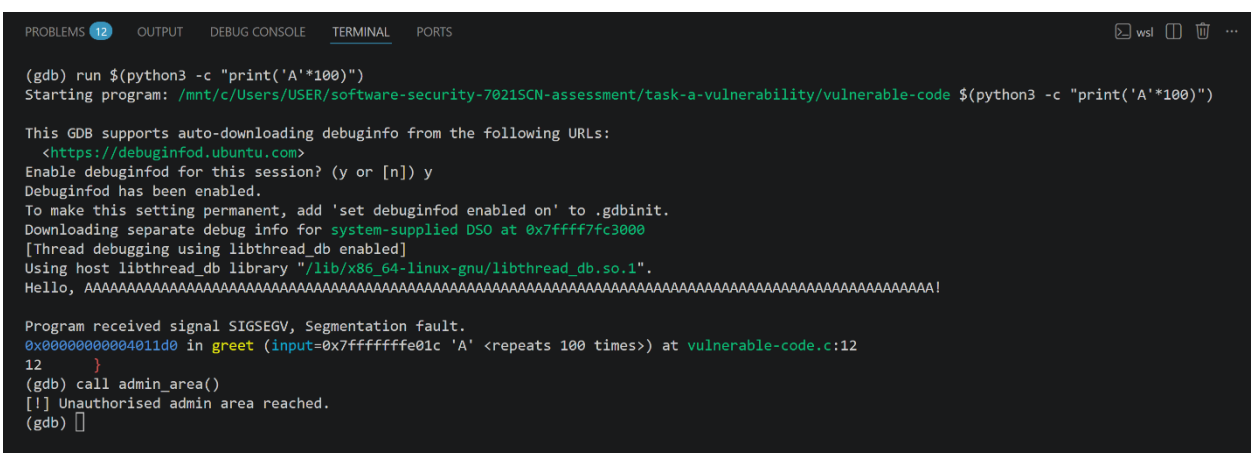


Figure 5: Stored XSS payload firing on the My Shop webapp (Task D evidence)

Four vulnerabilities were exploited manually: cookie forgery, SQL injection, stored XSS, and privilege escalation. OWASP ZAP returned 10 alerts including SQL injection, CSRF, and absent security headers. ZAP confirmed the SQL injection but missed cookie forgery and privilege escalation entirely. Both are logic flaws with no detectable syntax pattern. Automated scanners test what they can measure, not what they cannot anticipate. The most critical vulnerability in this application was invisible to every automated tool used.

Vulnerability Discovery and Remediation – Task A


PROBLEMS 12
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
wsl
(gdb) run $(python3 -c "print('A'*100)")
Starting program: /mnt/c/Users/USER/software-security-7021SCN-assessment/task-a-vulnerability/vulnerable-code $(python3 -c "print('A'*100)")
This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello, AAAA

The vulnerable C programme uses *strcpy* to copy user input into a 32-byte buffer with no length check. Passing 100 bytes caused an immediate segmentation fault. In GDB, the overflow redirected execution to *admin_area()*, a restricted function that should never be reachable. The fix replaces strcpy with *snprintf* and adds an explicit length check before copying. Compiler flags including *-fstack-protector-strong* and *-z noexecstack* provide runtime defence in depth. Fixing the root cause in source eliminates the vulnerability entirely. Compiler protections reduce exploitability, but an unsafe function call remains unsafe regardless of how the binary is compiled.

Repository link

https://github.com/OpeAkoki/software-security-7021SCN-assessment

AI Usage Table

Tool	How it was used in this assignment
Claude (Anthropic)	Paraphrasing, grammar checking, improving clarity of written sections, and suggesting structural improvements to documentation.

References

Anchore (2024) Grype: A vulnerability scanner for container images and filesystems. Available at: https://github.com/anchore/grype (Accessed: 23 April 2026).

Anchore (2024) Syft: CLI tool and library for generating a software bill of materials from container images and filesystems. Available at: https://github.com/anchore/syft (Accessed: 23 April 2026).

CycloneDX (2024) CycloneDX Specification Overview. Available at: https://cyclonedx.org/specification/overview/ (Accessed: 23 April 2026).

MITRE (2023) CWE-89: Improper Neutralisation of Special Elements used in an SQL Command ('SQL Injection'). Available at: https://cwe.mitre.org/data/definitions/89.html (Accessed: 23 April 2026).

MITRE (2023) CWE-121: Stack-based Buffer Overflow. Available at: https://cwe.mitre.org/data/definitions/121.html (Accessed: 23 April 2026).

National Institute of Standards and Technology (2022) Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities, SP 800-218. Available at:

https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf (Accessed: 23 April 2026).

OWASP (2021) *OWASP Top Ten*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 23 April 2026).

OWASP (2024) *OWASP Threat Dragon*. Available at: https://owasp.org/www-project-threat-dragon/ (Accessed: 23 April 2026).

OWASP (2024) *OWASP ZAP (Zed Attack Proxy)*. Available at: https://www.zaproxy.org/ (Accessed: 23 April 2026).

Pallets (2024) *Flask Documentation*. Available at: https://flask.palletsprojects.com/ (Accessed: 23 April 2026).

Semgrep (2024) *Semgrep Documentation*. Available at: https://semgrep.dev/docs/ (Accessed: 23 April 2026).